АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ ПРОФЕССИОНАЛЬНАЯ ОБРАЗОВАТЕЛЬНАЯ ОРГАНИЗАЦИЯ МОСКОВСКИЙ МЕЖДУНАРОДНЫЙ КОЛЛЕДЖ ЦИФРОВЫХ ТЕХНОЛОГИЙ «АКАДЕМИЯ ТОП»

Юридический адрес: 127254, г. Москва, вн. тер. г. муниципальный округ Бутырский, ул. Яблочкова, д. 8А,
тел.: +7(495)151-19-01, http://msk.top-academy.ru/college-ru
ИНН 7730265193 КПП 771501001 ОГРН 1217700253594

ДНЕВНИК

прохождении производственной практики по специальности

09.02.07 «Информационные системы и программирование»,

профессиональный модуль:

Осуществление интеграции программных модулей

Выполнил	Шаповалов Николай Викторович
Группа	9/3-РПО-23/54
Оценка	_____
Руководитель практики	Карево Александр Сергеевич

Практика пройдена в **Автономной некоммерческой организации профессиональной образовательной организации Московский международный колледж цифровых технологий «Академия ТОП»**

Дата прохождения с «13» апреля 2026 г. по «24» апреля 2026 г.

Москва, 2026

Учет выполненной работы

Срок выполнения	**Краткое содержание работы** (заполняется практикантом)	**Отметка о выполнении работы** (комментарии, подпись руководителя практики)
14.04.26 — 15.04.26	Проведён анализ текущих бизнес-процессов клуба настольных игр. Выявлены основные сущности и действия. Составлены Use Case диаграмма AS-IS, отражающая взаимодействие сотрудников и администратора с существующими журналами и Excel-таблицами, и Activity диаграмма AS-IS, описывающая последовательность операций при обслуживании клиента.	
15.04.26 — 16.04.26	Разработаны текстовые сценарии Use Case для всех ключевых сценариев использования системы. Составлен список нефункциональных требований на основе интервью с заказчиком. Построена ER-диаграмма будущей системы, с указанием всех атрибутов, первичных и внешних ключей, а также связей между таблицами.	
16.04.26 — 17.04.26	Разработана концептуальная архитектурная схема системы, отражающая клиент-серверное взаимодействие. Изучен шаблон технического задания. На основе анализа предметной области и интервью с заказчиком заполнены разделы ТЗ: «Общие сведения», «Цели и назначение развития системы», «Требования к системе». К документу приложены диаграммы описывающие устройство системы	
17.04.26 — 18.04.26	Разработана диаграмма потоков данных для системы бронирования. Диаграмма отражает движение данных между внешними сущностями, процессами и хранилищами. Определены входные и выходные потоки, обеспечивающие полный цикл работы приложения.	
18.04.26 — 19.04.26	Разработана UML-диаграмма классов системы бронирования. Диаграмма описывает структуру классов, их атрибуты, методы и взаимодействия. Определены способы передачи информации между	

	классами, включая создание заявок, обработку запросов администратором и оплату бронирований.	
19.04.26 — 20.04.26	Сформирована группа из 3 человек. Составлен список вопросов для интервью с заказчиком, охватывающий текущие процессы, планируемые улучшения и архитектурные предпочтения. Проведено интервью с заказчиком, ответы зафиксированы в таблице «вопрос–ответ». На основе полученных данных разработаны Use Case диаграмма TO-BE, отображающая всех актёров и их сценарии, а также Activity диаграмма TO-BE, описывающая поток операций сотрудника при обслуживании клиента.	
20.04.26 — 21.04.26	Разработан Веб-сайт на языке программирования PHP, реализованы сервисы для регистрации клиентов, управления номерами и заявками. Реализовано бронирование номеров с проверкой доступности дат. Добавлен функционал администратора: просмотр всех заявок, одобрение/отклонение, управление активностью и ценой номеров. Настроены уведомления: админу о новых заявках, клиенту о решении по заявке. Внедрена автоматическая отмена неоплаченных броней через 24 часа. Проведено тестирование всех сценариев работы.	
21.04.26 — 22.04.26	Освоены инструменты пошаговой отладки Visual Studio. Проведён анализ исходного кода системы хеширования. Выявлены и исправлены такие ошибки как: инициализация коллекций, некорректный формат хешей, регистрозависимое сравнение, выход за границы, отсутствие валидации значений. Реализована обработка исключений: проверка входных данных, работа с файлами, неподдерживаемые алгоритмы, некорректный формат данных, блокировка пользователя, целостность файлов, корректное освобождение ресурсов из памяти.	
22.04.26 — 23.04.26	Проведена систематическая проверка качества кода по чек-листу. Выявлены нарушения: отсутствие XML-документации у публичных методов, магические числа,	

	нерегистронезависимое сравнение хешей, дублирование кода преобразования в HEX. Все нарушения устранены: добавлены XML-комментарии, магические числа вынесены в именованные константы, дублирующий код вынесен в отдельные методы. Обеспечена null-безопасность, ресурсы обёрнуты в using. Заполнена таблица инспекции с указанием критичности замечаний.	
23.04.26 — 24.04.26	Разработан набор из 100 модульных тестов с подробными тест-кейсами и документацией на фреймворке xUnit для всех сервисов. Реализованы позитивные и негативные сценарии, проверка граничных условий, исключений, регистронезависимости, детерминированности, лавинного эффекта. Для изоляции тестов используются временные файлы. Покрытие кода составляет 80%. Все тесты успешно проходят.	

Студент-практикант _____ _____

(подпись) (Ф.И.О.)

1. Организация прохождения практики
   Автономная некоммерческая организация профессиональная образовательная организация Московский международный колледж цифровых технологий «Академия ТОП».
2. Основные сферы деятельности организации
   Обучение / учебная практика.
3. Материально-техническое обеспечение рабочего места
   Персональный компьютер с доступом в интернет, компьютерная мышь, клавиатура, операционная система Windows, Среда разработки Visual Studio Code, пакет офисных приложений LibreOffice.
4. Описание полученных на практике заданий:
   Проектирование архитектуры программного обеспечения в соответствии с требованиями.
   Разработка программного обеспечения с пользовательским интерфейсом
   Тестирование и отладка готового программного продукта в целях улучшения качества программного обеспечения
5. Описание деятельности относительно задания практики
   Составлял различные диаграммы на основе проведенного анализа предприятия для демонстрации его внутренней работы.
   Проектировал ER, DFD и прочие UML диаграммы, а так же архитектуру приложения. Разрабатывал веб-сайт с использыванием языка программирования PHP версии 8.2 и интеграцией реляционной базы данных MySQL. Занимался отладкой кода с помощью встроеного инструментария среды разработки, исправлял ошибки и отлавливал исключения. Инспектировал качество кода по чек-листу, а так же проводил автоматизированное тестирование с помощью тестов xUnit.
6. Результаты проделанной работы по каждой задаче
   Диаграммы Use Case, Activity AS-IS и TO-BE, а так же ER, DFD и UML

классов. Заполненное техническое задание с разделами и ссылками на диаграммы. Веб-сайт с функциями: регистрации, бронирования, одобрения/отклонения заявок на бронь и нотификацией. Отлаженный качественный код, прошедший инспекцию и . 100 модульных xUnit тестов, что в сумме покрывают 80% кода

7. Самоанализ проделанной работы

   Проявив аналитические способности и системный подход, выполнил все задания практики в срок и с высоким качеством.

   Изучены такие навыки как:

   - Анализ бизнес-процессов организации и детальное описание этих процессов на основе UML, DFD, ER и прочих диаграмм
   - Разработка технического задания на основе интервью с заказчиком.
   - Веб разработка, работа с PHP, реляционными базами данных MySQL, внедрение системы нотификации на основе событий
   - Анализ, отладка и инспекция кода готового продукта.
   - Автоматическое тестирование кода и написание модульных xUnit тестов.

Выполненные во время практики работы

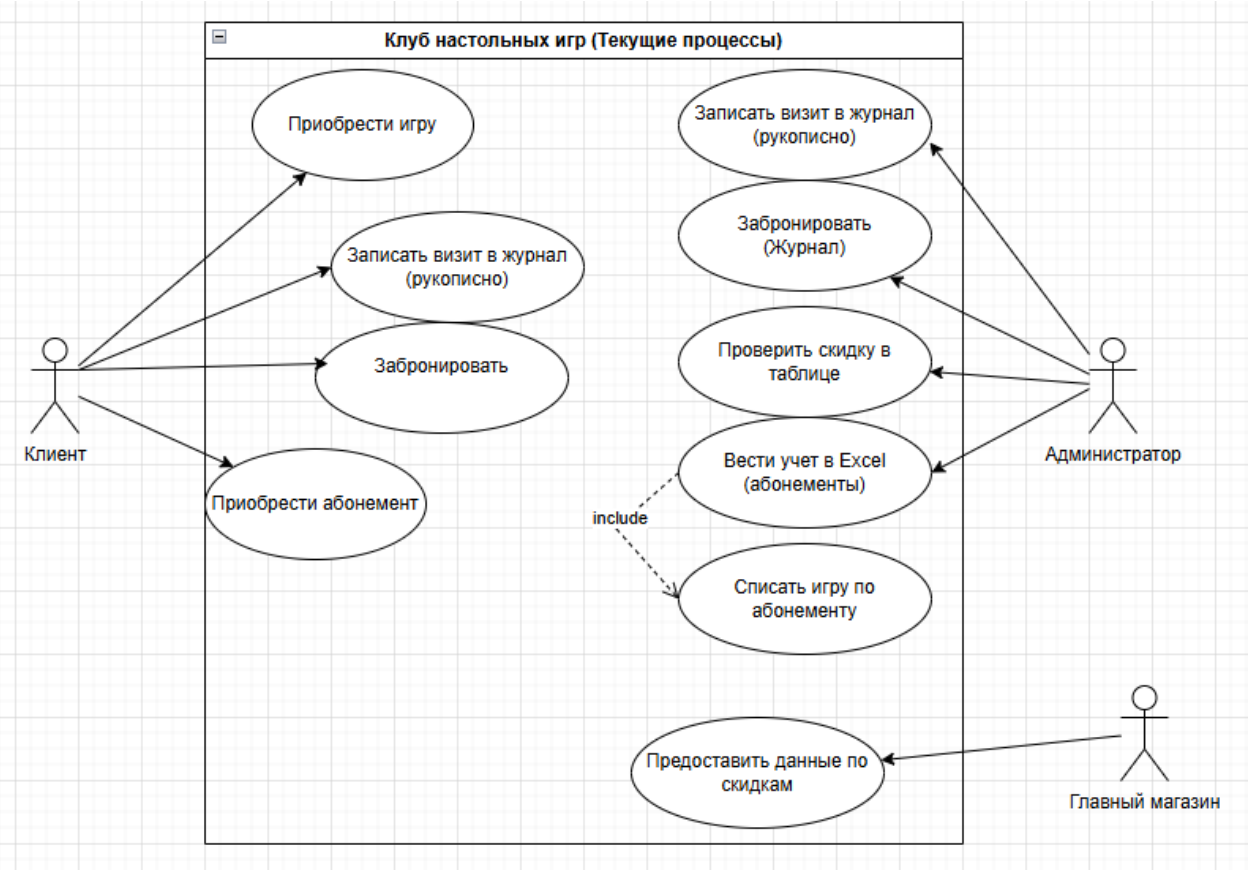

Рис. 1 - Use Case диаграмма AS-IS

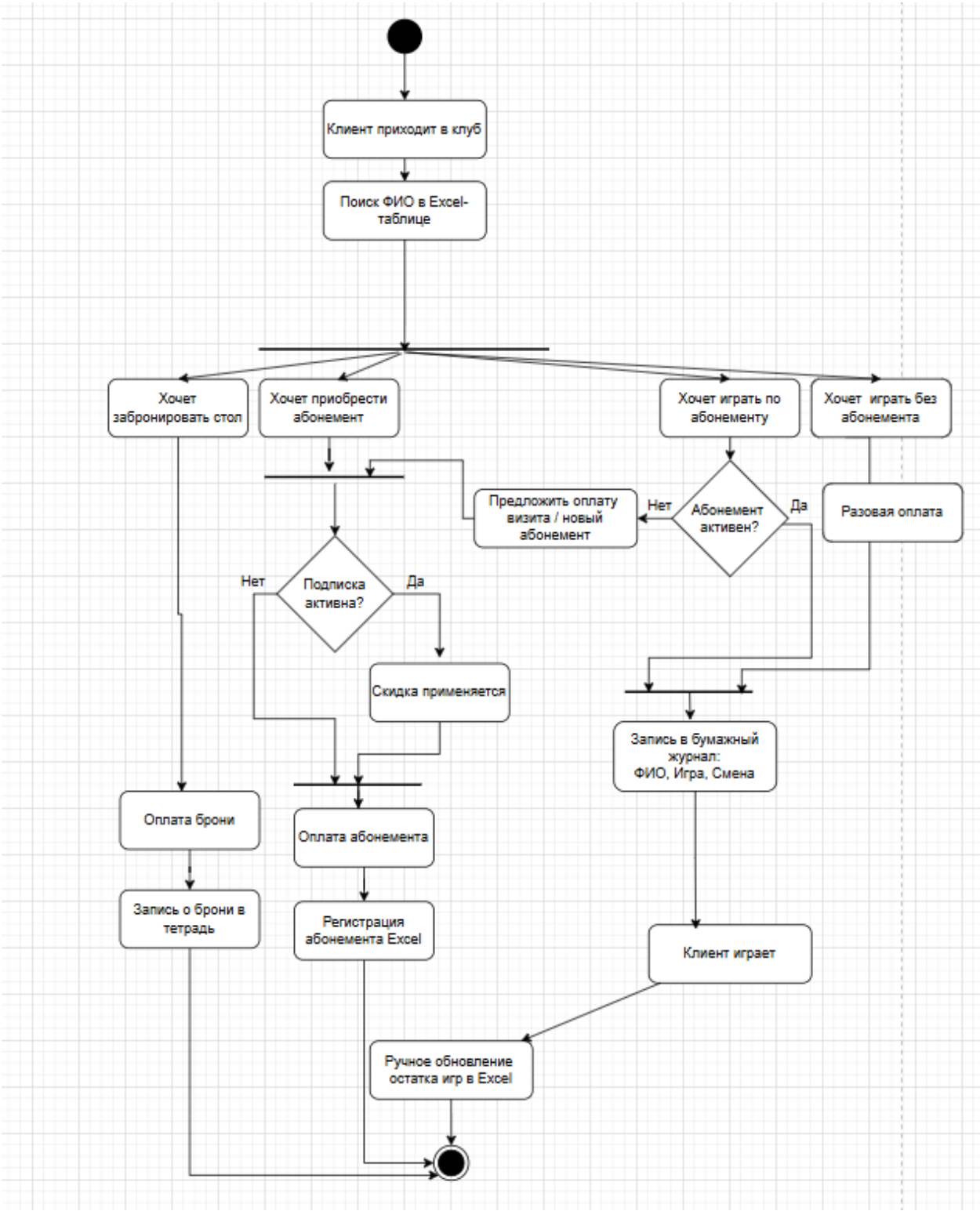

Рис. 2 - Activity диаграмма AS-IS

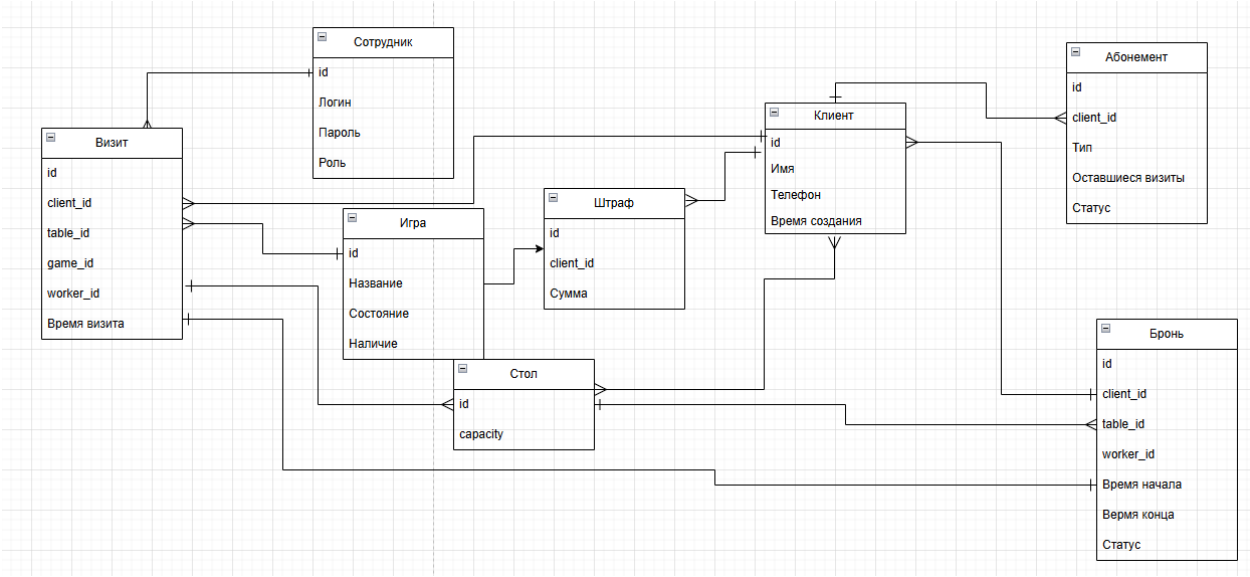

Рис. 3 - ER-диаграмма будущей системы

UC-1: Запросить данные о скидках в CRM

Актор: Администратор / Менеджер
Предусловие: Пользователь авторизован в CRM с ролью, имеющей доступ к справочнику скидок.

Основной поток:

1. Пользователь открывает раздел «Скидки и акции».

2. Система отображает список действующих скидок (постоянные, временные, персональные).

3. Пользователь может отфильтровать скидки по типу, размеру или сроку действия.

4. Система выводит детальную информацию по выбранной скидке (условия, процент, даты действия, применимые категории).

Альтернативные сценарии:

1. Нет скидок в БД: Система отображает сообщение «Скидки не найдены».

2. Срок действия скидки истёк: При запросе система помечает такую скидку как «неактивна» и по запросу позволяет её продлить или архивировать.

Рис. 4 - Use Case в текстовом формате

Список нефункциональных требований

Производительность и эффективность

Время отклика системы: Время выполнения простых операций (поиск клиента по ФИО, открытие карточки брони) не должно превышать 0,5 секунды при одновременной работе до 5 администраторов.

Генерация отчетов: Формирование «Общей системы отчетов» (аналитика за месяц по всем визитам и играм) должно занимать не более 3 секунд при объеме базы данных до 100 000 записей.

Пропускная способность: Система должна поддерживать корректную обработку данных при пиковой нагрузке до 50 новых записей о визитах в час без деградации производительности.

Надежность и доступность

Коэффициент доступности: Система должна быть доступна для работы не менее 99,5% времени в часы работы клуба (uptime).

Автоматизация бизнес-логики: Система должна автоматически переводить статус бронирования в «Отменено», если отметка о приходе клиента не была поставлена в течение 20 минут после назначенного времени start_time.

Целостность данных: При обрыве соединения в момент записи визита, транзакция должна полностью откатываться (механизм ACID в PostgreSQL), не допуская появления «висячих» записей без привязки к ID абонемента.

Отказоустойчивость: Время восстановления работоспособности после критического сбоя (при наличии бэкапа) не должно превышать 30 минут.

Рис. 5 - Список нефункциональных требований к системе

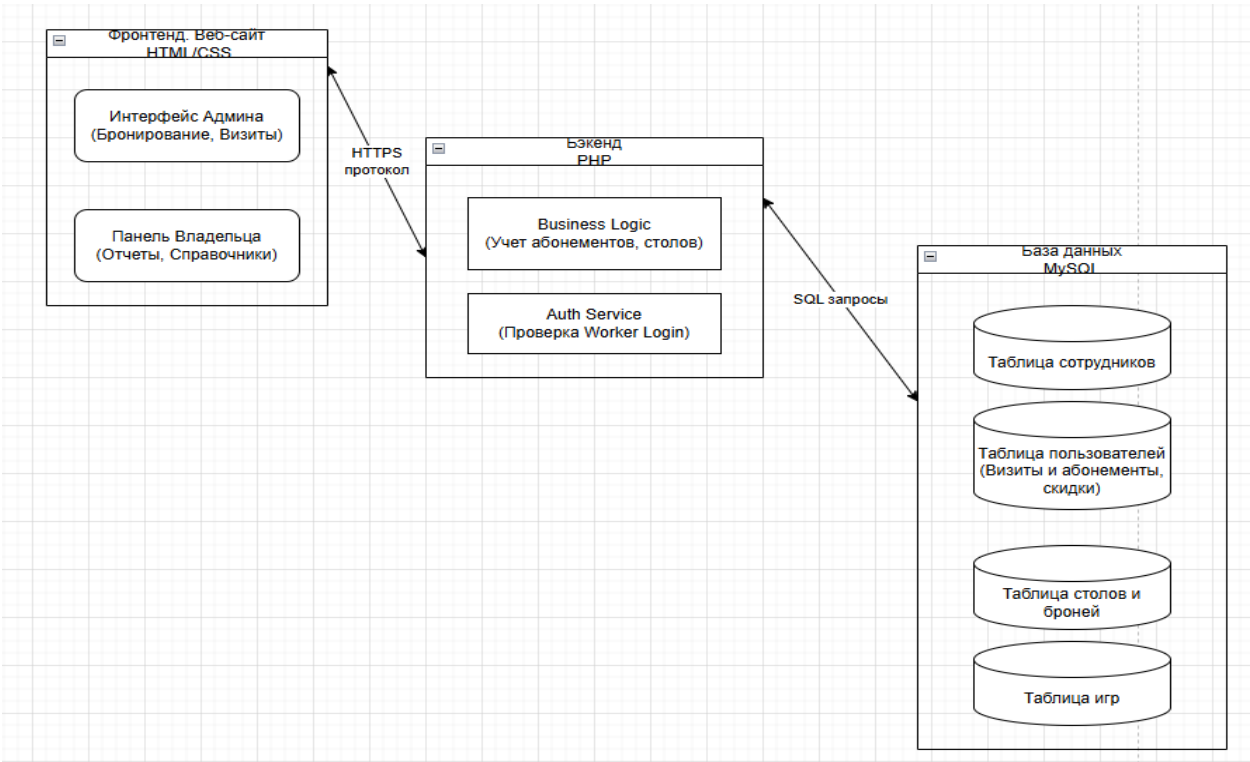

Рис. 6 - Концептуальная архитектурная схема системы

1 Общие сведения

ГОСТ 34.602— 2020

4.3 В разделе «Общие сведения» указывают следующее:

— полное наименование АС и ее условное обозначение;

— шифр темы при наличии;

— наименование организации — заказчика АС,

наименование организации-разработчика при наличии сведений о ней;

— перечень документов, на основании которых создается АС, кем и когда утверждены эти документы;

— плановые сроки начала и окончания работ по созданию АС;

— общие сведения об источниках и порядке финансирования работ.

Примечание — К документам, на основании которых или в соответствии с которыми создается АС, могут относиться, например, следующие:

— договорные документы на создание АС;

— нормативно-правовые и нормативно-технические документы, регламентирующие создание АС;

— техническое задание на создание ранее разрабатывавшейся АС.

Рис. 7 - Техническое задание

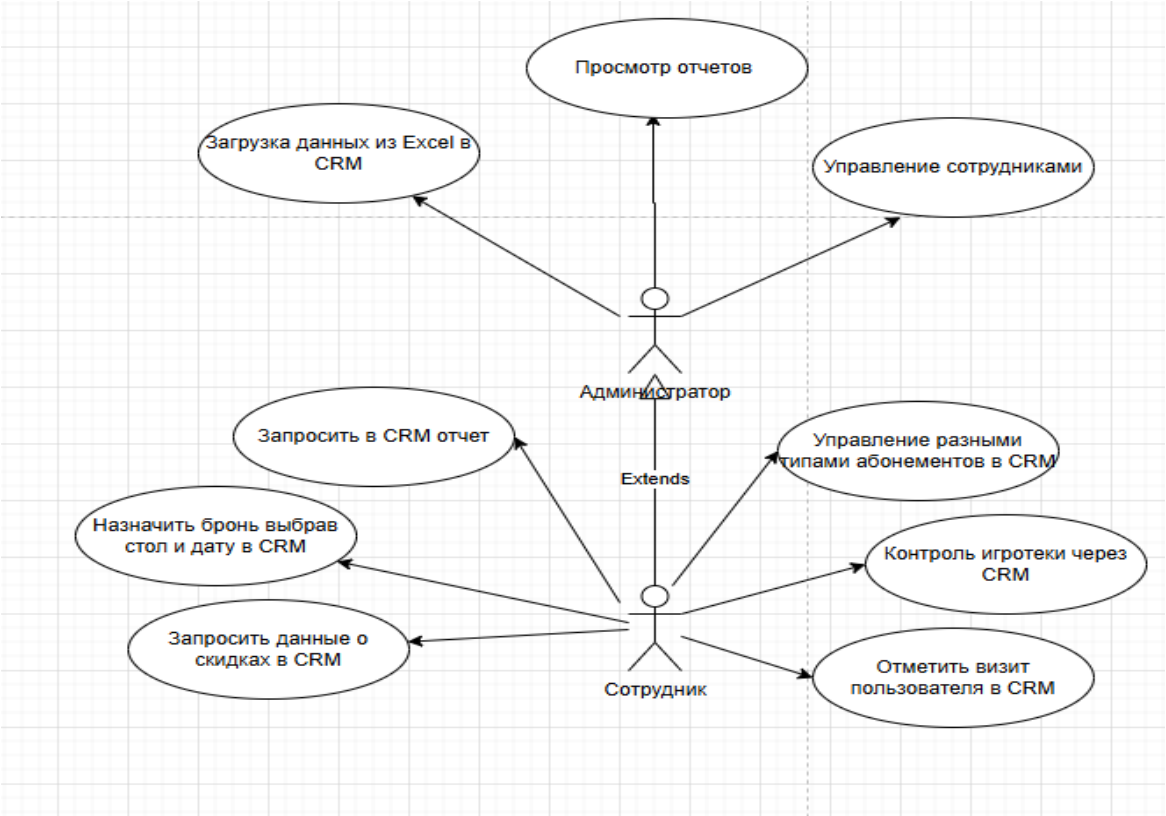

Рис. 8 - Use Case диаграмма TO-BE

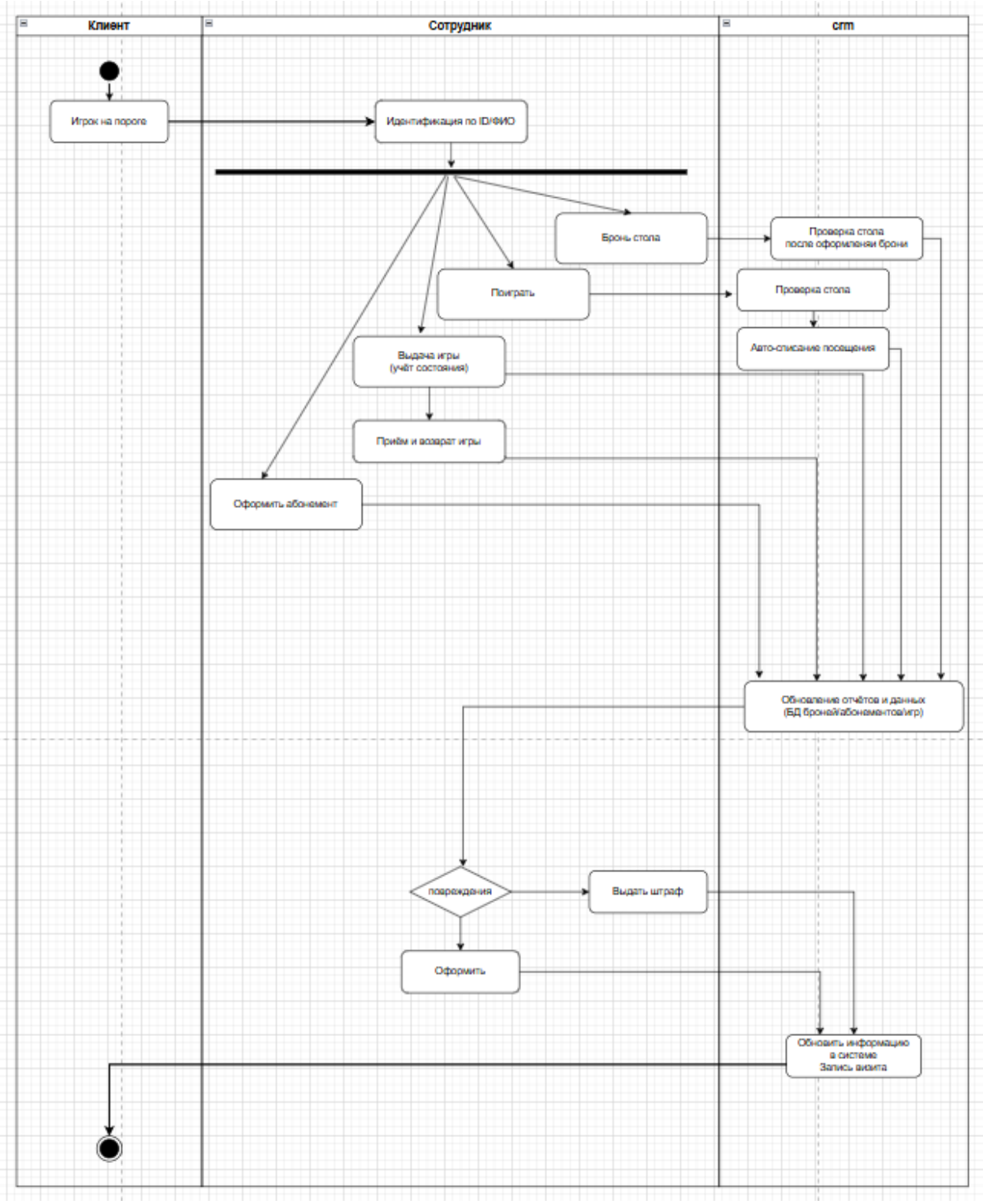

Рис. 9 - Activity диаграмма TO-BE

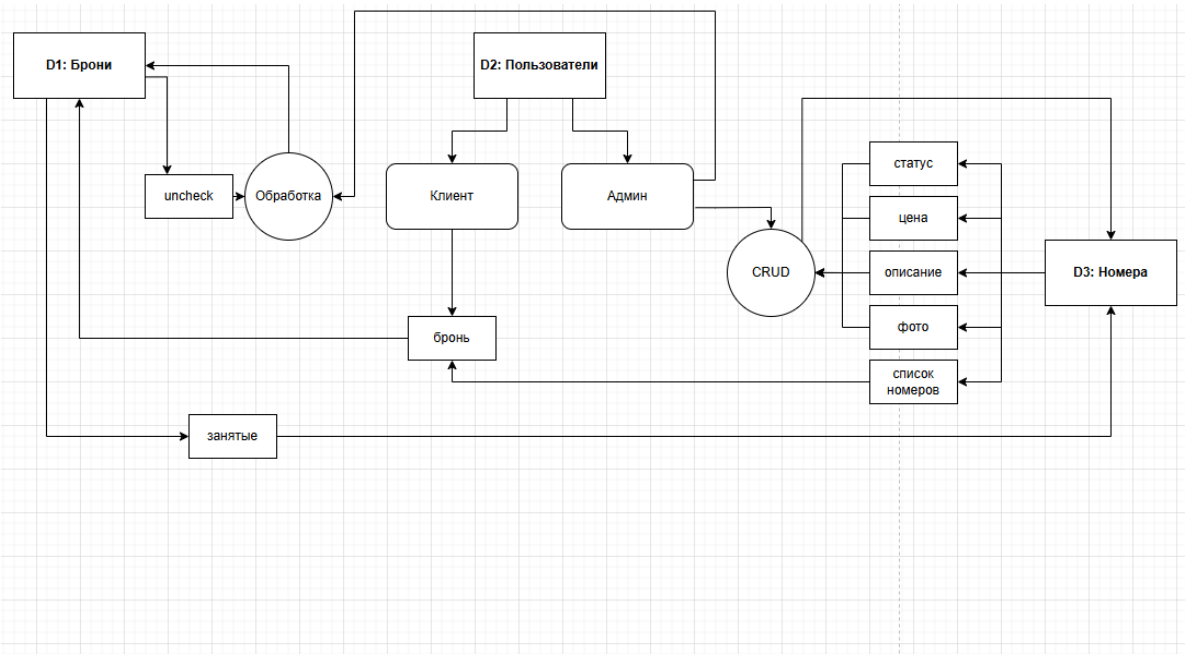

Рис. 10 - Диаграмма потоков данных (DFD) системы бронирования

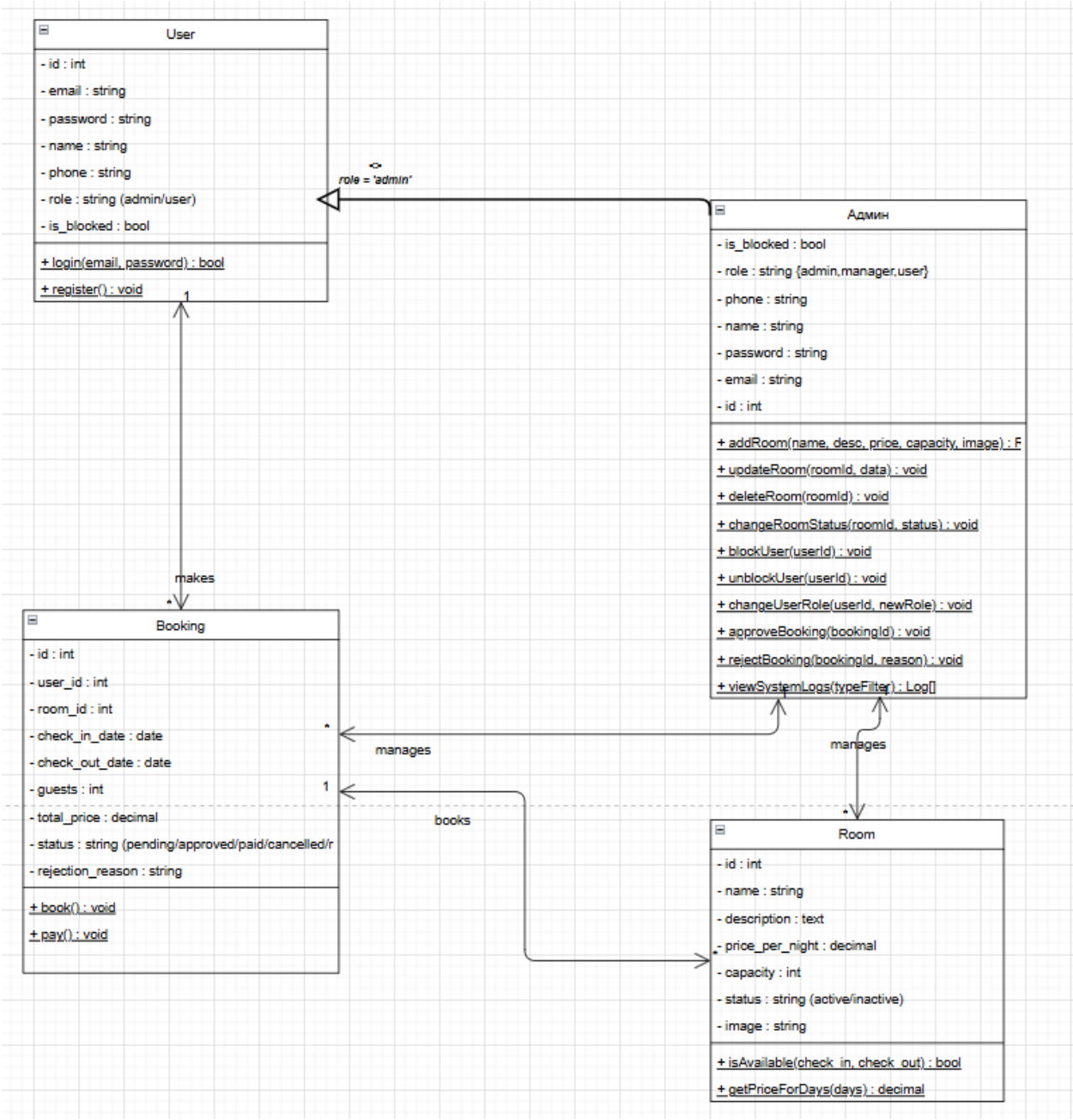
User
- id : int
- email : string
- password : string
- name : string
- phone : string
- role : string (admin/user)
- is_blocked : bool
+ login(email, password) : bool
+ register() : void
role = 'admin'
Admin
- is_blocked : bool
- role : string {admin,manager,user}
- phone : string
- name : string
- password : string
- email : string
- id : int
+ addRoom(name, desc, price, capacity, image) : F
+ updateRoom(roomId, data) : void
+ deleteRoom(roomId) : void
+ changeRoomStatus(roomId, status) : void
+ blockUser(userId) : void
+ unblockUser(userId) : void
+ changeUserRole(userId, newRole) : void
+ approveBooking(bookingId) : void
+ rejectBooking(bookingId, reason) : void
+ viewSystemLogs(typeFilter) : Log[]
1
makes
*
Booking
- id : int
- user_id : int
- room_id : int
- check_in_date : date
- check_out_date : date
- guests : int
- total_price : decimal
- status : string (pending/approved/paid/cancelled/r
- rejection_reason : string
+ book() : void
+ pay() : void
manages
manages
1
books
Room
- id : int
- name : string
- description : text
- price_per_night : decimal
- capacity : int
- status : string (active/inactive)
- image : string
+ isAvailable(check_in, check_out) : bool
+ getPriceForDays(days) : decimal

Рис. 11 - UML диаграмма классов системы бронирования

Вопрос	Ответ
Что именно администратор ищет в Excel после прихода клиента?	ФИО
Как сейчас вручную фиксируется «списание игры» по абонементу?	Сейчас нет учета игротенки
Кто и когда обновляет остаток игр в Excel?	Нет учета игротеки
Как проверяется скидка и кто её утверждает?	Скидки на абонементы и игры в таблице по праздникам
Сколько держат бронь, что если клиент опоздал и как происходит отмена брони	Опоздание на полчаса-звонок
Какие 2–3 процесса должны стать полностью автоматическими в первую очередь?(Например: списание игр, проверка скидки, уведомление об окончании абонемента)	Общая система отчетов, автоматический сбор информации. Автоматизация учета обработка пограничных значений удобный UI UX
Будет ли клиент взаимодействовать с системой до прихода в клуб? (Онлайн-бронь стола, просмотр остатка игр по абонементу, покупка/продление)	По телефону или в мессенджере, автоматизация тольлко внутренних процессов
Какие есть виды абонементов(Час два день)	Безлимит, 5 10 15 посещений, не переносим не продлеваем
Есть ли ограничение по количеству человек на столе	1 стол на 8 человек 2 стола на 4 человека 3 стола на 2 человека

Рис. 12 - Интервью с заказчиком (таблица в формате «вопрос-ответ»)

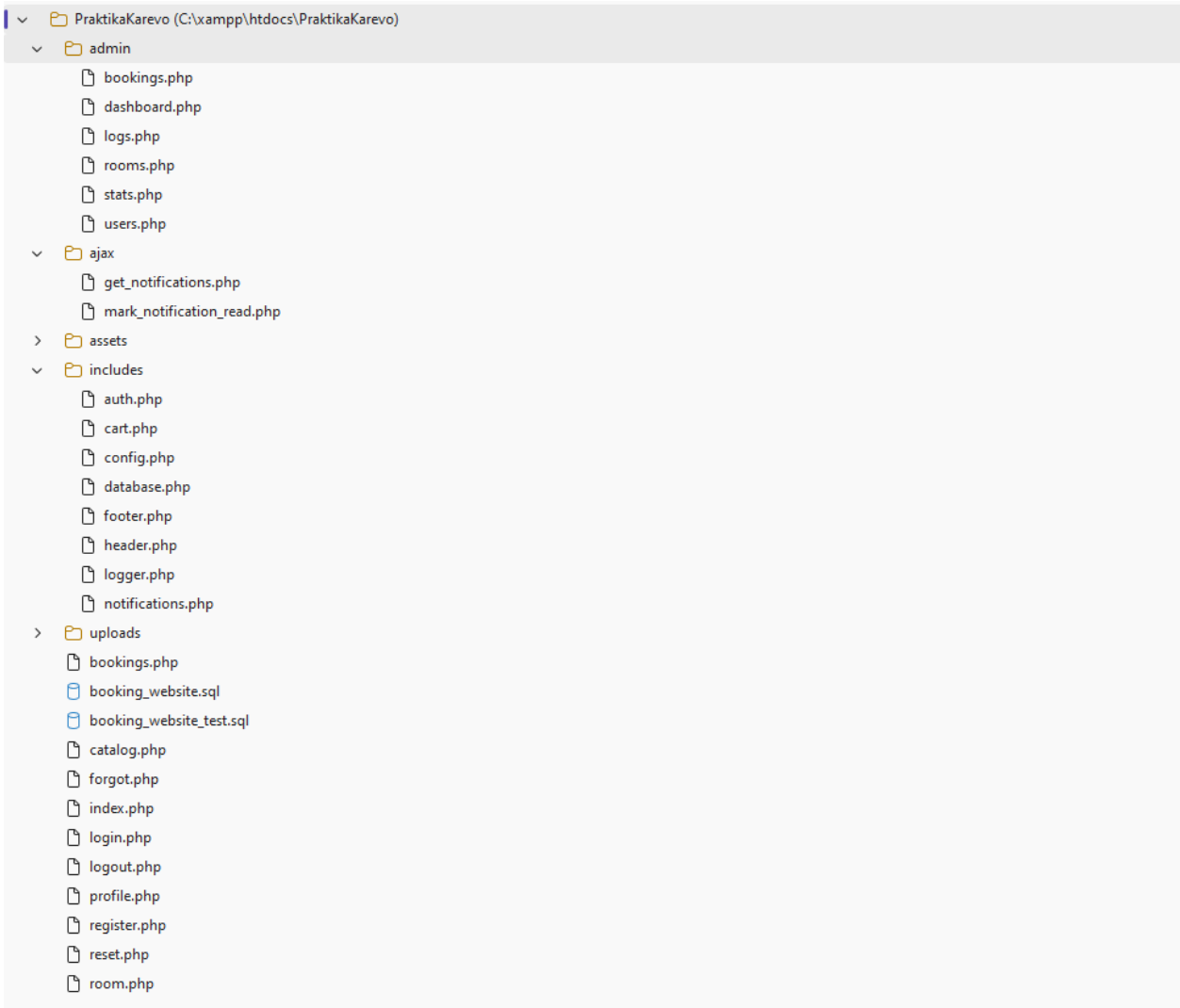

Рис. 13 - Структура проекта

index.php

```php
<?php require_once 'includes/database.php'; ?>
<!DOCTYPE html>
<html lang="ru">
<head>
<meta charset="UTF-8">
<title>Гостиница - бронирование номеров</title>
<link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
<?php include 'includes/header.php'; ?>

<h1>Добро пожаловать в нашу гостиницу</h1>
<p>Комфортные номера, отличный сервис. Забронируйте номер на нужные даты.</p>

<div class="rooms-grid">
<?php
// Показываем последние 4 активных номера
$stmt = $pdo->prepare("SELECT * FROM rooms WHERE status='active' ORDER BY created_at DESC LIMIT 4");
$stmt->execute();
$rooms = $stmt->fetchAll(PDO::FETCH_ASSOC);

foreach($rooms as $room):
    $img = !empty($room['image']) ? 'uploads/' . $room['image'] : 'assets/images/no-image.jpg';
?>
<div class="room-card">
    <img class="room-img" src="<?= htmlspecialchars($img) ?>" alt="<?= htmlspecialchars($room['name']) ?>">
    <div class="room-info">
        <h3><?= htmlspecialchars($room['name']) ?></h3>
        <p>Вместимость: до <?= (int)$room['capacity'] ?> чел.</p>
        <p class="room-price"><?= number_format($room['price_per_night'], 0, '.', ' ') ?> ₽ / ночь</p>
        <a href="room.php?id=<?= $room['id'] ?>" class="btn">Подробнее и бронь</a>
    </div>
</div>
<?php endforeach; ?>
</div>

<p style="margin-top: 2rem;"><a href="catalog.php" class="btn">Смотреть все номера →</a></p>

<?php include 'includes/footer.php'; ?>
</body>
</html>
```

Рис. 14 - Пример кода: файл index.php

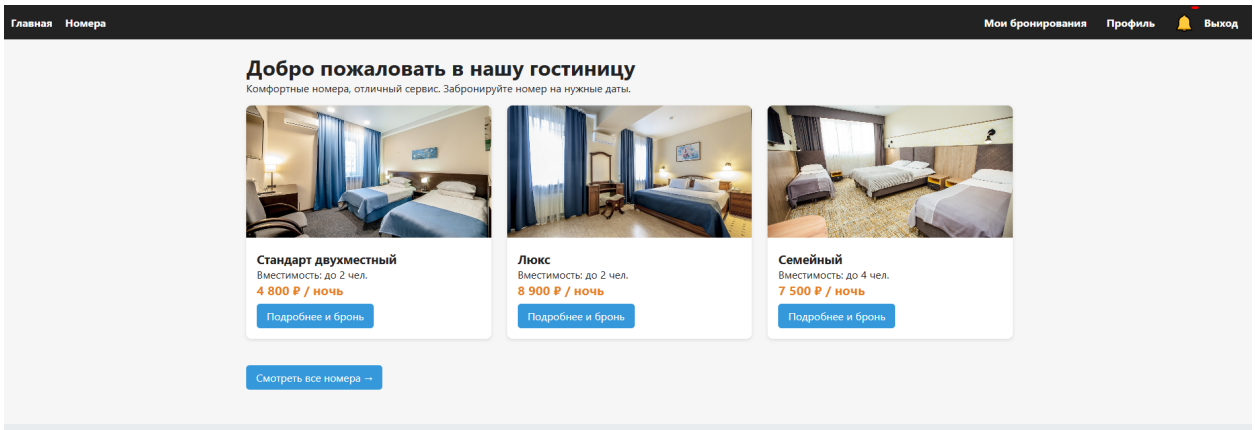

Рис. 15 - Интерфейс веб-сайта: Главная страница

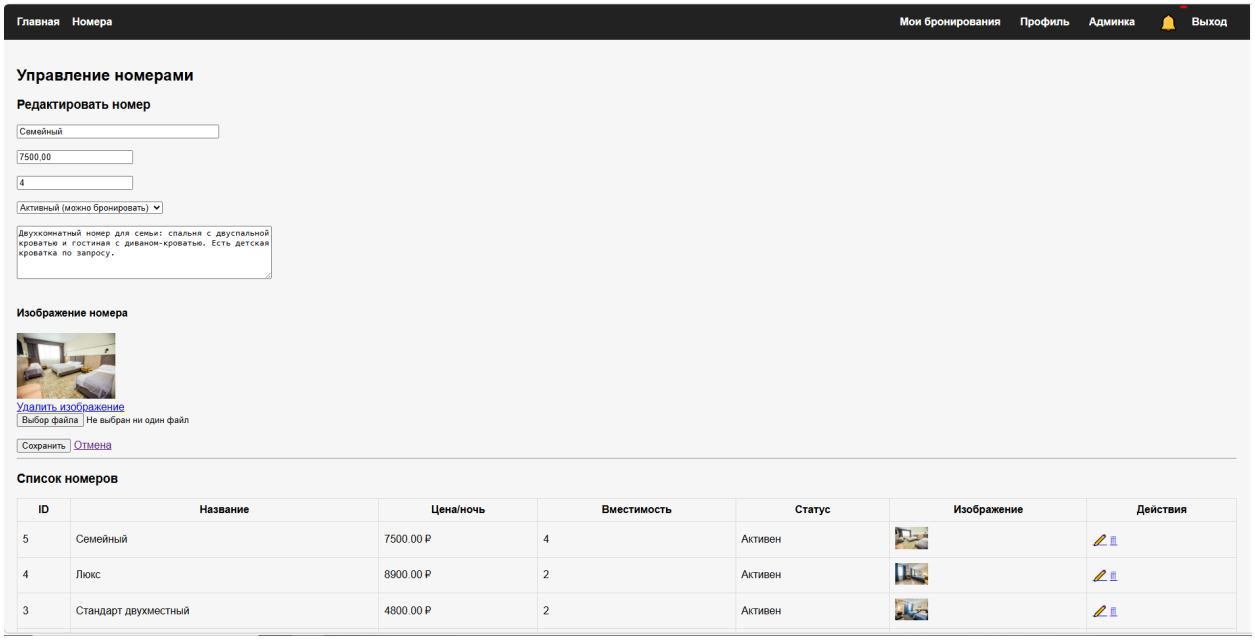

Рис. 16 – Интерфейс веб-сайта:Страница управления номерами

№	Файл	Метод	Строка(до исправления)	Тип ошибки	Описание
1	HashService.cs	поле _logs	12	Инициализация	Не инициализирован список логов → NullReferenceException при вызове LogOperation
2	HashService.cs	ComputeSha512	27	Формат хеша	Возвращает Base64, в то время как ComputeSha256 возвращает HEX
3	HashService.cs	ComputeMd5	36	Формат хеша	Возвращает HEX с дефисами (например, A1-B2-C3)
4	HashService.cs	ComputeHmacSha256	52	Формат хеша	Возвращает lowercase, остальные методы – uppercase
5	HashService.cs	VerifyHash	44-50	Логическая	Для SHA512 использует ComputeSha512, который возвращает Base64, а сравнивает с HEX
6	HashService.cs	LogOperation	75-86	Инициализация	Добавляет в _logs, но _logs может быть null
7	HashService.cs	GetLogs	89	Инициализация	Возвращает _logs (null)
8	FileIntegrityService.cs	RegisterFile	23	Отсутствие проверки	Не проверяет существование файла
9	FileIntegrityService.cs	ComputeFileHash	46-50	Преобразование	Передаёт Convert.ToBase64String(content) вместо байтов
10	FileIntegrityService.cs	VerifyFile	36	NullReference	Не проверяет, найдена ли запись (record может быть null)
11	FileIntegrityService.cs	GetTamperedFiles	60	Выход за границы	Условие i <= _records.Count → IndexOutOfRangeException
12	FileIntegrityService.cs	GetTamperedFiles	60-64	Производительность	В цикле вызывает VerifyFile, который каждый раз читает файл с диска (не ошибка, но неэф
13	UserService.cs	CountLockedUsers	89	Логическая	count =+ 1 → присваивание, а не инкремент
14	UserService.cs	RegisterUser	28	Отсутствие проверки	Не проверяет пароль на null или пустоту
15	UserService.cs	VerifyPassword	40	Отсутствие проверки	Не проверяет username и password на null
16	UserCredential.cs	RegisterFailedAttempt	35	Логическая	Блокировка при FailedAttempts > 5, но должно быть >= 5 (иначе блокировка только на 6-й п
17	StorageService.cs	SaveCredentials	13-24	Отсутствие директории	Не создаёт папку Data, если её нет
18	StorageService.cs	LoadCredentials	26-44	Отсутствие проверки	Не проверяет существование файла → FileNotFoundException
19	StorageService.cs	LoadCredentials	35-38	Потеря данных	Не восстанавливает CreatedAt и LastLoginAt (не парсятся)
20	StorageService.cs	SaveFileRecords	49-57	Потеря данных	Не сохраняет поле LastCheckedAt
21	StorageService.cs	LoadFileRecords	60-77	Потеря данных	Не восстанавливает LastCheckedAt
22	StorageService.cs	SaveLogs	79-89	Отсутствие директории	Не создаёт папку Data
23	Program.cs	Main	70-71	Логическая	Вызов SaveLogs до того, как были добавлены логи (логи пусты)
24	Program.cs	Main	75-78	Логическая	Загруженные пользователи и записи не передаются обратно в сервисы
25	Program.cs	Main	65,70,71	Отсутствие директории	Каталог Data может не существовать при записи файлов

Рис. 17 - Журнал ошибок

```
// ------------------ Работа со строками (текстовые данные) ------------------

/// <summary>Вычисляет SHA-256 для строки (UTF-8). Возвращает HEX в верхнем регистре, 64 символа.</summary>
/// <param name="input">Входная строка.</param>
/// <returns>Хеш в виде HEX-строки.</returns>
/// <exception cref="ArgumentNullException">Если input == null.</exception>
Ссылок: 7
public string ComputeSha256(string input)
{
    if (input == null) throw new ArgumentNullException(nameof(input));
    try
    {
        byte[] bytes = Encoding.UTF8.GetBytes(input);
        return ComputeSha256(bytes);
    }
    catch (Exception ex)
    {
        LogError("ComputeSha256", "SHA256", input.Length > 20 ? input.Substring(0, 20) : input, ex);
        throw;
    }
}
```

Рис. 18 - Обработка исключений

№	Файл	Метод / Свойство	Строка	Замечание
1	FileRecord.cs	Все поля, конструктор	7–27	Нет XML-документации и описания
2	FileRecord.cs	Все поля, конструктор	7–30	Нет XML-документации и описания
3	UserCredential.cs	RegisterFailedAttempt	31	Константа MaxFailedAttempts отсутствует, используется магичес
4	UserCredential.cs	RegisterFailedAttempt	31	Логика: блокировка при FailedAttempts > 5 вместо >= 5
5	UserCredential.cs	Все поля	9–28	Нет XML-документации и описания
6	FileIntegrityService.cs	RegisterFile, VerifyFile, GetTamperedFiles	-	Нет проверки/обработки ошибок, отсутствие try/catch, не создаёт
7	FileIntegrityService.cs	ComputeFileHash	35	Нельзя использовать Convert.ToBase64String(content), обрабатыв
8	FileIntegrityService.cs	GetTamperedFiles	45	В цикле условие i <= _records.Count, а должно быть i < _records.C
9	FileIntegrityService.cs	VerifyFile	22	Нет проверки на null record, если файл не зарегистрирован
10	FileIntegrityService.cs	VerifyFile	30	Нет try/catch для FileNotFoundException, возможен NullReference
11	HashService.cs	Все методы конвертации строки в байты	-	Дублирование кода преобразования строки в байты
12	HashService.cs	ComputeSha256, ComputeSha512, ComputeMd5	10–45	Магические числа (размер соли, длины хеша, путь к файлам не в
13	HashService.cs	ComputeHmacSha256	32	Нет проверки на пустоту ключа
14	HashService.cs	ComputeSha512, ComputeMd5, ComputeHmacSha256	21,28,34	Несогласованный формат вывода (SHA256 — HEX, SHA512 — I
15	HashService.cs	Все публичные методы	-	Нет XML-документации и описания
16	HashService.cs	VerifyHash, VerifyWithSalt	43,73	Для сравнения хеша используется ==, а не string.Equals(..., Ordina
17	HashService.cs	Все методы	-	Не оборачивает IDisposable-объекты в using
18	HashService.cs	HashWithSalt	54	Используется порядок salt+input, что не соответствует остальной
19	HashService.cs	ComputeFileHash	35	Нет поддержки SHA512, хотя указан в чек-листе
20	StorageService.cs	Сохранение/загрузка данных	6–72	Нет проверки на null для параметров методов
21	StorageService.cs	SaveCredentials, SaveFileRecords, SaveLogs	9,40,63	Нет создания директории, если она не существует, при сохранен
22	StorageService.cs	SaveCredentials, SaveFileRecords, SaveLogs	-	Нет обработки IOException — падение при ошибке записи
23	StorageService.cs	LoadCredentials, LoadFileRecords	21,53	Нет проверки существования файла при загрузке
24	StorageService.cs	LoadCredentials/LoadFileRecords	27,55	Нет FormatException при некорректной строке; возможна потеря/
25	StorageService.cs	Save/LoadFileRecords	42,7	Не сохраняется и не восстанавливается LastCheckedAt для FileRe
26	UserService.cs	RegisterUser, VerifyPassword, ChangePassword	-	Нет проверки на пустое/Null имя пользователя и пароль перед ис
27	UserService.cs	RegisterUser	19	Не проверяется существование коллекции _users (может быть не
28	UserService.cs	CountLockedUsers	74	Описка: count =+ 1, должно быть count += 1 или count++

Рис. 19 - Инспекция кода

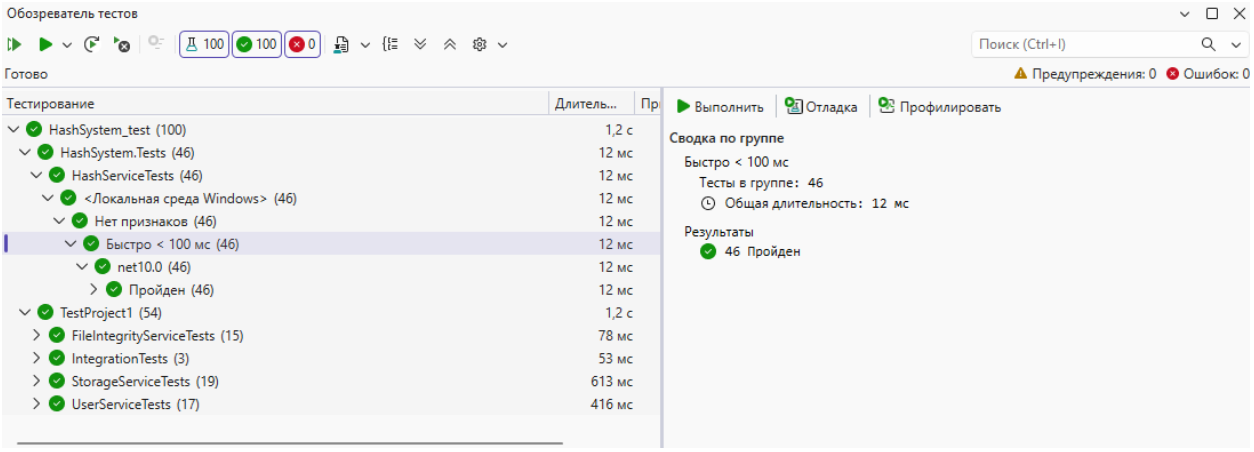

Рис. 20 - Набор модульных тестов

ID теста	Класс	Метод	Сценарий	Статус(Пройден/Не пройден)
TC-01	HashService	ComputeSha256_ValidInput_Returns64CharHex	Корректный хеш	Пройден
TC-02	HashService	ComputeSha256_NullInput_ThrowsArgumentNullException	Передача null	Пройден
TC-03	HashService	ComputeMd5_ValidInput_Returns32CharHex	Корректный хеш MD5	Пройден
TC-04	HashService	ComputeHmacSha256_ValidKeyAndMessage_Returns64CharHex	Корректный HMAC	Пройден
TC-05	HashService	ComputeHmacSha256_EmptyKey_ThrowsArgumentNullException	Пустой ключ	Пройден
TC-06	HashService	VerifyHash_ValidMatch_ReturnsTrue	Хеши совпадают	Пройден
TC-07	HashService	VerifyHash_InvalidMatch_ReturnsFalse	Хеши не совпадают	Пройден
TC-08	HashService	GenerateSalt_Length16_ReturnsNonEmptyBase64	Генерация соли	Пройден
TC-09	HashService	VerifyHash_UnsupportedAlgorithm_ThrowsNotSupportedException	Неподдерживаемый алгоритм	Пройден
TC-10	HashService	HashWithSalt_ValidInput_ReturnsDifferentHashesForDifferentSalts	Разные соли → разные хеши	Пройден
TC-11	HashService	VerifyHash_WrongHashLength_ThrowsInvalidDataException	Неверная длина хеша	Пройден
TC-12	HashService	GenerateSalt_NegativeLength_ThrowsArgumentOutOfRangeException	Отрицательная длина	Пройден
TC-13	UserService	RegisterUser_NewUser_UserAdded	Регистрация нового пользователя	Пройден
TC-14	UserService	RegisterUser_Duplicate_ThrowsInvalidOperationException	Дубликат имени	Пройден
TC-15	UserService	VerifyPassword_CorrectPassword_ReturnsTrue	Верный пароль	Пройден
TC-16	UserService	VerifyPassword_WrongPassword_ReturnsFalse	Неверный пароль	Пройден
TC-17	UserService	VerifyPassword_FiveFailedAttempts_LocksAccount	Блокировка после 5 неудач	Пройден
TC-18	UserService	ChangePassword_ValidOldPassword_PasswordChanged	Смена пароля (верный старый)	Пройден
TC-19	UserService	ChangePassword_InvalidOldPassword_ThrowsInvalidOperationException	Смена пароля (неверный старый)	Пройден
TC-20	UserService	RegisterUser_EmptyUsername_ThrowsArgumentNullException	Пустое имя пользователя	Пройден
TC-21	UserService	RegisterUser_EmptyPassword_ThrowsArgumentNullException	Пустой пароль	Пройден
TC-22	FileIntegrityService	RegisterFile_ExistingFile_RecordAdded	Регистрация существующего файла	Пройден
TC-23	FileIntegrityService	VerifyFile_UnchangedFile_ReturnsTrue	Файл не изменён	Пройден
TC-24	FileIntegrityService	VerifyFile_ChangedFile_ThrowsDataMisalignedException	Файл изменён	Пройден
TC-25	FileIntegrityService	VerifyFile_NotRegistered_ThrowsInvalidOperationException	Файл не зарегистрирован	Пройден

Рис. 21 – Отчёт по тестам